

结构化编程 II — 变量

面向对象编程 · 2026 · 南京大学

| 刘钦

Outline

本讲是结构化编程三讲的第二讲（数据层面），上一讲讲设计方法，下一讲讲行为（方法）。

- **为什么需要变量？** — 动机：状态与赋值
- **赋值的代价** — 引用透明性、环境模型、函数式 vs 命令式
- **类型系统** — 约束变量、保证代码安全
- **变量的属性** — 名字、地址、生存期、作用域、默认值
- **变量的实战角色** — 封装、局部变量角色、属性字段角色

为什么需要变量？

从状态到赋值

变量

变量的三种视角

物理视角

计算机的数据
存储单元

地址 | 单元内容

逻辑视角

软件模型中的
抽象数据单元

`int sum = 0;`

语义视角

类的属性
方法的状态

`sum` 是方法的
临时状态

问题：银行账户

有一个银行账户，我存了100万，取了30万，问能不能再取80万？

- $100 - 30$ 是否大于等于80？ → 需要记住 **balance**
- $100 - 30$ 意味着什么？ → **状态**

从状态到变量

- 现实对象的状态 → 计算对象的状态 → 用程序设计语言的**符号名字**来模拟
- 由于现实对象的状态随时间要变化 → 符号名字的值也要变 → **变量**
- 怎么变? → **赋值语句**

赋值的代价

引用透明性 · 环境模型 · 函数式 vs 命令式

赋值的代价

- 有赋值
 - `(define (make-simplified-withdraw balance)`
 - `(lambda (amount)`
 - `(set! balance (- balance amount))` //赋值
 - `balance))`
- 无赋值
 - `(define (make-decrements balance)`
 - `(lambda (amount)`
 - `(- balance amount))`

赋值打破了引用透明性！

引用透明性：同一的东西可以相互替换

```
(make-simplified-withdraw 25)  
-> (set! balance (- 25 20) 25)  
-> balance到底等于5 还是25?
```

但与所有状态都必须**显式地操作**和**传递额外的参数**的方式相比，通过引进赋值和将状态隐藏在局部变量的技术，我们能以一种**更模块化**的方式构造系统。

命令式程序的缺陷

- 函数式

- `(define (factorial n)`
 - `(define (iter product counter)`
 - `(if (> counter n)`
 - `product`
 - `(iter (* counter product) (+ counter 1))`
 - `(iter 1 1))`

- 命令式

- `(define (factorial n)`
 - `(let ((product 1)`
 - `(counter 1))`
 - `(define (iter)`
 - `(if (> counter n)`
 - `product`
 - `(begin (set! product (* counter product))`
 - `(set! counter (+ counter 1))`
 - `(iter))`
 - `(iter))`

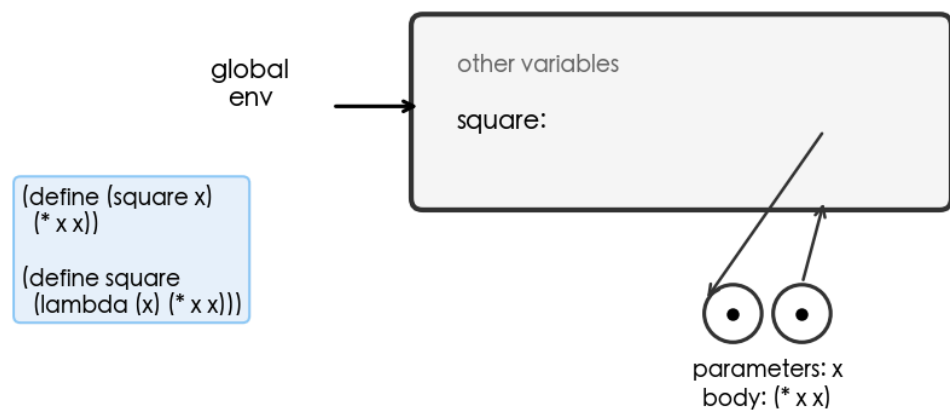
赋值的代价 — 小结

带有"赋值"的程序将强迫人们去考虑赋值的**"相对顺序"**, 以保证每个语句所用的是被修改变量的正确版本。

接下来用**环境模型**来理解赋值如何改变求值过程。

创建过程

求值的环境模型 - 创建过程



应用过程

```
(define (square x) (* x x))  
; 等价于  
(define square (lambda (x) (* x x)))  
  
(square 5)
```

调用 `(square 5)` 时创建新环境 E1，绑定 `x: 5`，在 E1 中求值 `(* x x)`。

无赋值过程

```
(define (square x) (* x x))  
(define (sum-of-squares x y)  
  (+ (square x) (square y)))  
(define (f a)  
  (sum-of-squares (+ a 1) (* a 2)))  
  
(f 5)
```

`(f 5)` $\rightarrow$ E1: `a:5` $\rightarrow$ `(sum-of-squares 6 10)` $\rightarrow$ E2: `x:6, y:10` $\rightarrow$ `(square 6)` E3 + `(square 10)` E4

环境之间无约束，完全通过参数传递数值

有赋值过程

```
(define (make-withdraw balance)
  (lambda (amount)
    (if (>= balance amount)
        (begin (set! balance (- balance amount))
                balance)
        "Insufficient funds"))))

(define W1 (make-withdraw 100))
```

第一步: `make-withdraw` 绑定到全局环境

第二步: `(make-withdraw 100)` 创建闭包, E1 中 `balance: 100`

第三步: W1 指向该闭包

有赋值过程 – (1)

```
(define W1 (make-withdraw 100))  
(W1 50)
```

调用 `(W1 50)` 时，在 E1 中查找 `balance`（值为 100），`(>= 100 50)` 为真，执行 `(set! balance (- 100 50))`。

E1 中 `balance` 被修改为 50。

Here is the balance that will be changed by the set!

有赋值过程 – (2)

调用完成后，E1 中 `balance: 50`。

下次调用 `(w1 25)` 时，`balance` 已经是 50，而非 100。

状态被封装在闭包的环境中。

有赋值过程 – 新环境

```
(define W1 (make-withdraw 100))  
(W1 50)  
(define W2 (make-withdraw 100))  
(W2 50)
```

W1 和 W2 各自有**独立的环境** (E1 和 E2)，各自维护独立的 `balance`。

环境之间有约束，新环境维护局部状态变量，`this`指针

内部定义

```
(define (sqrt x)
  (define (good-enough? guess)
    (< (abs (- (square guess) x)) 0.001))
  (define (improve guess)
    (average guess (/ x guess)))
  (define (sqrt-iter guess)
    (if (good-enough? guess)
        guess
        (sqrt-iter (improve guess))))
  (sqrt-iter 1.0))
```

`(sqrt 2)` → E1: `x:2, good-enough?, improve, sqrt-iter` 都定义在 E1 中, 可以访问 `x`。

用变动数据做模拟

```
(define x (list (list a b) c d))  
(define y (list e f))
```

- `(set-car! x y)` → x 的 car 指向 y
- `(set-cdr! x y)` → x 的 cdr 指向 y

改变也就是赋值

用过程表达有序对

```
(define (cons x y)
  (define (dispatch m)
    (cond ((eq? m 'car) x)
          ((eq? m 'cdr) y)
          (else (error "Undefined operation - CONS" m))))
  dispatch)

(define (car z) (z 'car))
(define (cdr z) (z 'cdr))
```

改变就是赋值

将变动数据对象实现为使用赋值和局部状态的过程

```
(define (cons x y)
  (define (set-x! v) (set! x v))
  (define (set-y! v) (set! y v))
  (define (dispatch m)
    (cond ((eq? m 'car) x)
          ((eq? m 'cdr) y)
          ((eq? m 'set-car!) set-x!)
          ((eq? m 'set-cdr!) set-y!)
          (else (error "Undefined operation - CONS" m))))
  dispatch)
```

函数式 vs 命令式

函数式 vs 命令式

	函数式	命令式
解决问题思路	计算	模拟
实现的思路	递归	迭代
数据传递	参数	变量
传递方式	参数的代入	赋值

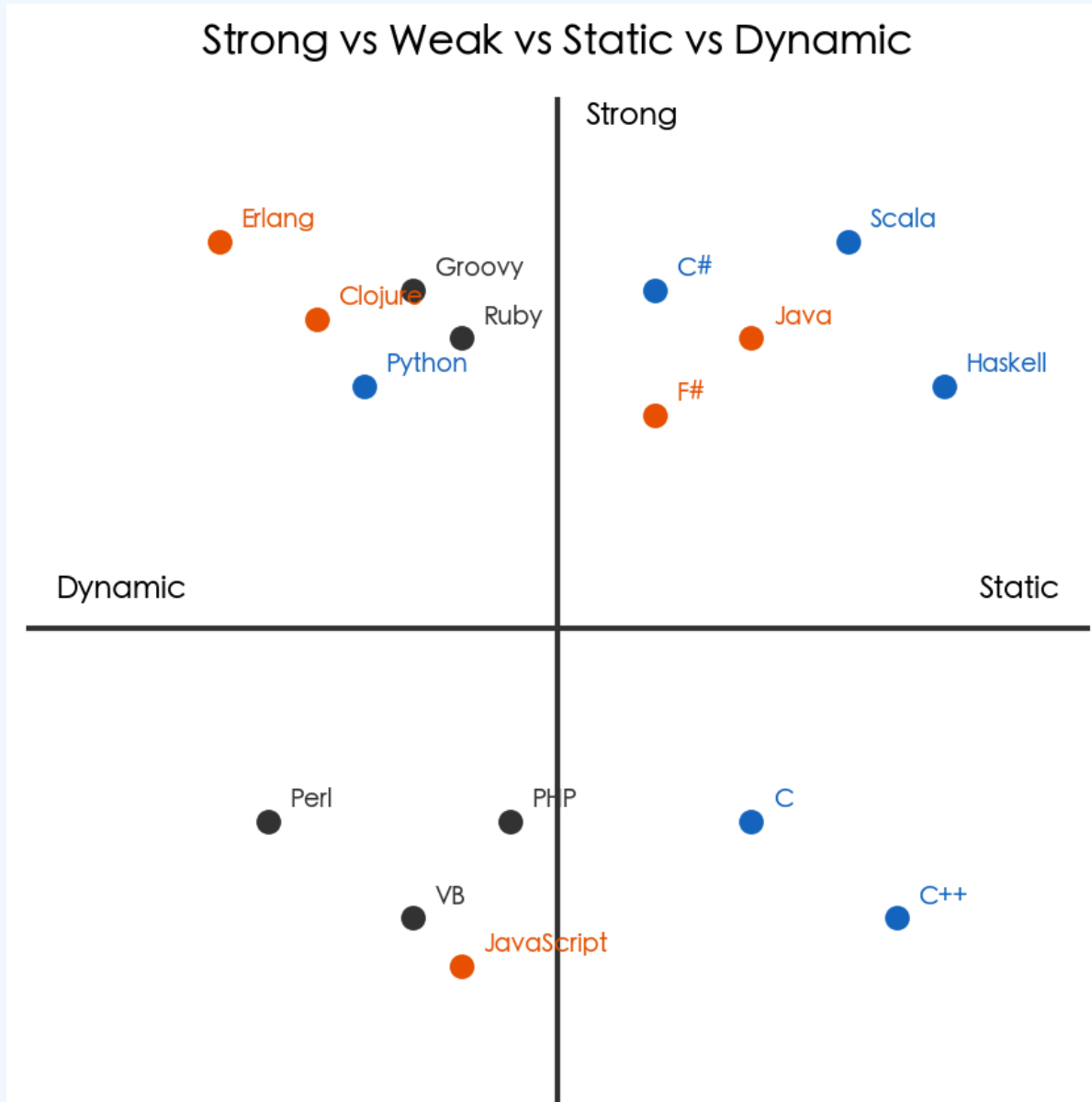
类型系统

约束变量 · 保证代码安全

Types, Values, and Variables

- The Java programming language is a **statically typed** language, which means that every variable and every expression has a type that is known at compile time.
- The Java programming language is also a **strongly typed** language, because types limit the values that a variable can hold or that an expression can produce, limit the operations supported on those values, and determine the meaning of the operations.

Strong vs Weak vs Static vs Dynamic



举个例子

- 在 Python 中执行 `test = '666' / 3` 你会在运行时得到一个 `TypeError` 错误，相当于运行时排除了 `untrapped error`，因此 Python 是动态类型，强类型语言。
- 在 JavaScript 中执行 `var test = '666' / 3` 你会发现 `test` 的值变成了 222，因为这里发生了隐式转换，因此 JavaScript 是动态类型，弱类型的。更为夸张的是 `[] == ![]` 在 JavaScript 中返回的是 `true`。
- 在 Java 中执行 `int[] arr = new int[10]; arr[0] = '666' / 3;` 你会在编译时期得到一个语法错误，这说明 Java 是静态类型的。执行 `int[] arr = new int[10]; arr[11] = 3;` 你会在运行时得到数组越界的错误 (`trapped error`)，这说明 Java 通过自身的类型系统排除了 `untrapped error`，因此 Java 是强类型的。
- 而 C 与 Java 类似，也是静态类型的，但是对于 `int test[] = {1, 2, 3}; test[4] = 5;` 这样的代码 C 语言是没办法发现你的问题的，因此这是 `untrapped error`，因此我们说 C 是弱类型的。

类型系统的价值

	弱类型	强类型
静态	C/C++：编译快，但缓冲区溢出、野指针	Java：编译期+运行期双重保护
动态	JavaScript：灵活但隐式转换陷阱多	Python：运行时类型安全，但无编译期检查

Java 的选择：静态 + 强类型 = 在编译期尽可能多地发现错误，在运行期兜底剩余错误，绝不让错误静默通过。

Types of Java

Types of Java

Primitive Types

numeric types

integral: byte, short,
int, long, char

floating: float, double

boolean type

Reference Types

class types

interface types

array types

special type: null

Types of Java

- The values of a reference type are references to objects.
- All objects, including arrays, support the methods of class Object.
- String literals are represented by String Object

The types of Java

- 基础数据类型 (Primitive types)
 - 来源于物理视角
 - 预定义好的、基础的数据类型
- 引用类型 (Reference types)
 - 来源与语义视角
 - 对象的遥控器

byte(8) · short(16) · int(32) · long(64) · reference (bit depth not relevant)

基础数据类型 (Primitive Types)

- 不用其他类型来定义的数据类型被称为基本数据类型
- Java是完全面向对象的，但简单数据类型不是——这是为性能做出的妥协
- 因为Java可移植性的要求，所有的数据类型都有一个严格定义的范围（不像C，int的大小取决于平台）

类型	字节	范围
byte	1	-128 ~ 127
int	4	±21亿
long	8	±9.2×10 ¹⁸
float	4	IEEE 754 单精度
double	8	IEEE 754 双精度
char	2	Unicode
boolean	-	true / false

引用变量

- There is actually no such thing as an object variable
- There's only an object reference variable
- An object reference variable holds bits that represent a way to access an object
- It doesn't hold the object itself, but it holds something like a pointer. Or an address.
- Except, in Java we don't really know what is inside a reference variable. We do know that whatever it is, it represents one and only one object. And the JVM knows how to use the reference to get the object.

值 (Value)

- 变量的值 = 与变量关联的**抽象存储单元**的内容
 - 物理上 float 可能占 4 字节，逻辑上看作**一个**抽象单元
- 变量与值的绑定 = **初始化**过程（涉及运算符、表达式）
- **右值** (rvalue)：变量当前持有的值
- **左值** (lvalue)：变量的存储位置（地址）

类型系统的陷阱：溢出

```
class Test {  
    public static void main(String[] args) {  
        int i = 1000000;  
        System.out.println(i * i);           // -727379968 (溢出!)  
        long l = i;  
        System.out.println(l * l);           // 1000000000000  
        System.out.println(20296 / (l - i)); // ArithmeticException  
    }  
}
```

- This program produces the output:
 - **-727379968** (32-bit overflow)
 - **1000000000000** (long multiplication)
- and then encounters an ArithmeticException in the division by `l - i`, because `l - i` is zero.

类型系统的陷阱：浮点数

- IEEE 754: `float` (32-bit) / `double` (64-bit)
- 特殊值: `+0.0`, `-0.0`, `Infinity`, `-Infinity`, `NaN`

```
double d = 1e308;  
System.out.println(d * 10);      // Infinity (溢出)  
System.out.println(0.0 / 0.0);   // NaN  
System.out.println(0.1 + 0.2);   // 0.30000000000000004 (精度丢失)
```

NaN 的诡异行为：

- `NaN == NaN` → `false`!
- `x != x` 为 `true` 当且仅当 `x` 是 `NaN`
- 所有比较运算符对 `NaN` 返回 `false`

教训：金融计算用 `BigDecimal`，判断 `NaN` 用 `Double.isNaN(x)`

三目运算符与类型提升

```
public class Conditional {  
    public static void main(String args[]) {  
        int x = 4;  
        System.out.println("value is " + ((x > 4) ? 99.9 : 9));  
    }  
}
```

- value is 9.0
- 三目运算符中：第二个表达式和第三个表达式中如果都为基本数据类型，整个表达式的运算结果由容量高的决定。99.9是double类型而9是int类型，double容量高。

类型系统：代码的安全网

"Well-typed programs cannot go wrong." — Robin Milner, 1978

类型系统在不同阶段拦截不同类型的错误：

阶段	拦截方式	示例
编译期	类型检查	<code>int[] arr; arr[0] = "hello";</code> → 编译错误
运行期	trapped error	<code>arr[11] = 3;</code> → <code>ArrayIndexOutOfBoundsException</code>
逃逸	untrapped error	C 语言 <code>test[4] = 5;</code> → 未定义行为，无报错

Java 的类型系统目标：消灭所有 untrapped error——要么编译期报错，要么运行期抛异常，绝不静默出错。

泛型与类型安全

```
// 没有泛型 (Java 1.4 之前) - 运行时才发现类型错误
ArrayList list = new ArrayList();
list.add("hello");
list.add(42);           // 编译通过!
String s = (String) list.get(1); // 运行时 ClassCastException!
```

```
// 有泛型 - 编译期就拦截类型错误
ArrayList<String> list = new ArrayList<>();
list.add("hello");
list.add(42);           // 编译错误! type mismatch
```

泛型让类型检查从运行时前移到编译时——越早发现错误，修复成本越低。

变量的属性

名字 · 地址 · 生存期 · 作用域 · 默认值

Name — 好名字的重要性

- 名字是用来标识程序中某些实体的字符串
- 好的名字是提高程序可读性的一项关键要素，名字要尽可能地具体

糟糕的变量名字：

```
x = x - xx;  
xxx = aretha + SalesTax(aretha);  
x = x + LateFee(x1, x) + xxx;  
x = x + Interest(x1, x);
```

良好的变量名字

```
balance = balance - lastPayment;  
monthlyTotal = NewPurchases + SalesTax(newPurchases);  
balance = balance + LateFee(customerID, balance) + monthlyTotal;  
balance = balance + Interest(customerID, balance);
```

- Java标识符：以字母/ `_` / `$` 开头，区分大小写，不含空格
- 采用任何一项命名规则都要好于没有规则！

Java命名规则

- 变量(Variables)
- 除了变量名外，所有实例，包括类，类常量，均采用大小写混合的方式，第一个单词的首字母小写，其后单词的首字母大写。变量名不应以下划线或美元符号开头，尽管这在语法上是允许的。
- 例如： `char c; int i; float myWidth;`

常量

- 类常量的声明，应该全部大写，单词间用下划线隔开。例如
- `static final int MIN_WIDTH = 4;`
- `static final int MAX_WIDTH = 999;`
- `static final int GET_THE_CPU = 1;`

保留关键字

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

地址

- 变量的位置
- 变量的地址是与这个变量相关联的地址。然而这种关联关系却不是看上去那么简单。在许多语言中，程序中的相同名字可以在不同的时间，与不同的地址相关联。
 - 例如，如果子程序有一个局部变量（当调用子程序时，它从运行时栈分配而来），不同的调用可能会导致变量有一个不同的地址。从某种意义上说，它们是同一变量的不同实例。
- 多个变量可以具有同一地址当用多个变量名来访问单个存储地址时，这些变量名就成为别名。别名使用有损于可读性，因为它允许通过给一个变量赋值来改变另一个变量的值。

生存期

- 变量所绑定的存储单元必须取自一个现有存储单元的"池"。这个过程就成为存储空间分配。存储空间解除分配则是将已经与变量解除绑定的存储单元重新放回到这个池中的过程。
- 变量的生存期是该变量被绑定于某一特定存储地址的时间。因此变量的生存期开始于将变量绑定到一个特定存储单元的时刻，而结束于该变量从这个存储单元上解除绑定之时。
- 为了分析变量的存储绑定，可以较方便地根据标量（非结构性的）变量的生存期将它们分成四种类型，分别被称为静态变量、栈动态变量、显性堆动态变量和隐性堆动态变量。

The difference between instance and local variables

① **Instance** variables are declared **inside a class** but not within a method.

```
class Horse {  
    private double height = 15.2;  
    private String breed;  
    // more code...  
}
```

② **Local** variables are declared **within a method**.

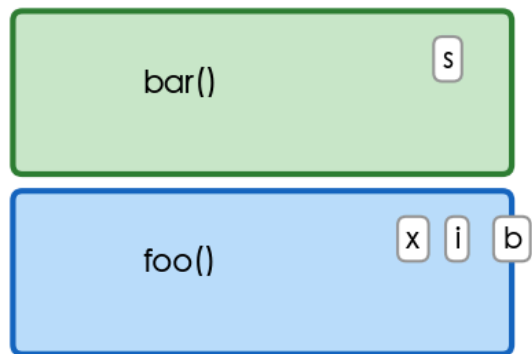
```
class AddThing {  
    int a;  
    int b = 12;  
    public int add() {  
        int total = a + b; // total 是局部变量  
        return total;  
    }  
}
```

局部变量（栈空间）的生存期

局部变量(栈空间)的生存期

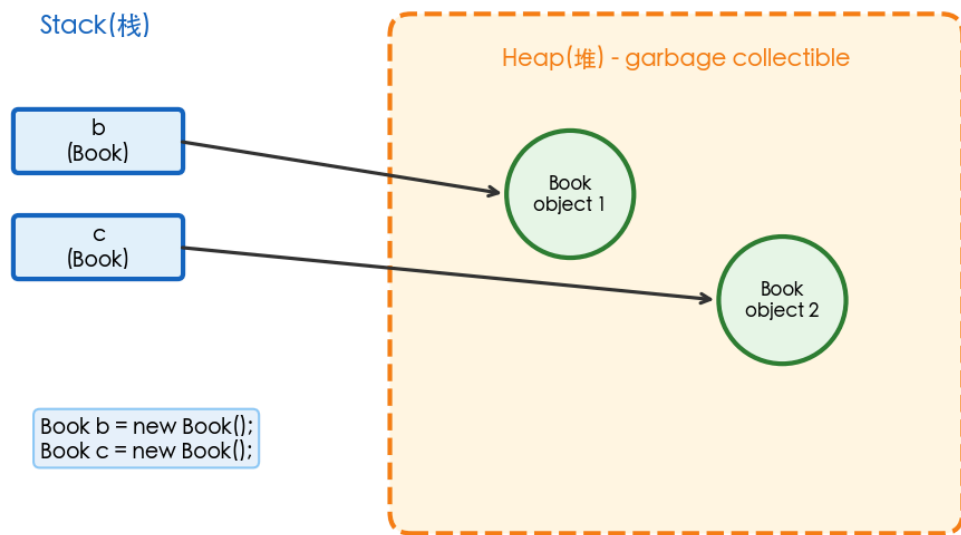
变量和方法栈的生存期一致

栈顶方法正在执行



成员变量（堆空间）的生存期

引用变量的声明与对象创建(堆空间)



垃圾回收

- `Book b = new Book(); Book c = new Book();`
 - 2 个引用, 2 个对象
- `Book d = c;`
 - 3 个引用, 2 个对象 (c 和 d 指向同一对象)
- `c = b;`
 - 3 个引用, 2 个对象 (**Object 1 被废弃, eligible for GC**)
- `c = null;`
 - Object 2 still has active reference (b), **not eligible for GC**
 - Active References: 1, null References: 1, Reachable Objects: 1, Abandoned: 1

What is the value before you initialize a variable?

```
class PoorDog {
    private int size;
    private String name;
    public int getSize() { return size; }
    public String getName() { return name; }
}

public class PoorDogTestDrive {
    public static void main(String[] args) {
        PoorDog one = new PoorDog();
        System.out.println("Dog size is " + one.getSize());
        System.out.println("Dog name is " + one.getName());
    }
}

// Output: Dog size is 0
//          Dog name is null
```

Instance variables 默认值

Instance variables - 默认值

Type	Default Value
integers	0
floating points	0.0
booleans	false
references	null

You don't have to initialize instance variables, because they always get a default value.

Local variables MUST be initialized before use!

```
class Foo {  
    public void go() {  
        int x;  
        int z = x + 3; // Won't compile!!  
    }  
}
```

```
% javac Foo.java  
Foo.java:4: variable x might not have been initialized  
    int z = x + 3;  
1 error
```

Local variables do NOT get a default value! The compiler complains if you try to use a local variable **before** the variable is initialized.

作用域

- 作用域：变量**可见**的语句范围
- 局部变量：声明于子程序或程序块内部
- 非局部变量：不在当前块中声明，但对其可见

案例 1:

```
int num = 0, i, j;
for (i = 1; i <= 10; i++) {
    for (int j = 1; j <= i; j++) { /* 错误：变量j已经声明过 */
        if (i + j == 10)
            num++;
    }
}
```

案例

```
class Test {  
    int k = 0;  
    void f() {  
        int k = 1;  
        int m = k;      /* m的值为1 */  
        int n = this.k; /* n的值为0 */  
    }  
}
```

变量的实战角色

封装 · 局部变量角色 · 属性字段角色

Encapsulating the GoodDog class

```
class GoodDog {  
    private int size;  
  
    public int getSize() { return size; }  
    public void setSize(int s) { size = s; }  
  
    void bark() {  
        if (size > 60) {  
            System.out.println("Woof! Woof!");  
        } else if (size > 14) {  
            System.out.println("Ruff! Ruff!");  
        } else {  
            System.out.println("Yip! Yip!");  
        }  
    }  
}
```

Make the instance variable **private**. Make the getter and setter methods **public**. Even though the methods don't really add new functionality, you can come back and make a method safer, faster, better.

GoodDogTestDrive

```
class GoodDogTestDrive {  
    public static void main(String[] args) {  
        GoodDog one = new GoodDog();  
        one.setSize(70);  
        GoodDog two = new GoodDog();  
        two.setSize(8);  
        System.out.println("Dog one: " + one.getSize());  
        System.out.println("Dog two: " + two.getSize());  
        one.bark();  
        two.bark();  
    }  
}
```

Any place where a particular value can be used, a method call that returns that type can be used. Instead of:

`int x = 3 + 24;` you can say: `int x = 3 + one.getSize();`

变量类型

- 局部变量
 - 方法内部
 - Stack
- 属性字段(成员变量)
 - 对象内部
 - Heap
- 静态变量
 - 全局 (类)
 - Code区

访问

- 直接访问

- `x=10;`
- 内部使用者

- 间接访问

- `position.x=10;`
- `position.setX(10);`
- 外部使用者

角色 – 局部变量

- 局部变量
 - 收集器 → `result`
 - 计数 → `count`
 - 解释 → `top, left`
 - 复用 → `now`
 - 元素 → `each`

解释 top left 案例

```
int top = ...;  
int left = ...;  
int height = ...;  
int width = ...;  
return new Rectangle(top, left, height, width);
```

复用 now 案例

```
// 原始版本（每次循环都调用系统时间）
for (Clock each : getClocks())
    each.setTime(System.currentTimeMillis());

// 优化版本（复用 now 变量）
long now = System.currentTimeMillis();
for (Clock each : getClocks())
    each.setTime(now);
```

元素 each 案例

```
broadcast() {  
    for (Source eachSender : getSenders())  
        for (Source eachReceiver : getReceivers())  
            ...  
}
```

属性字段

- **Helper** — 用于存放其它对象的引用
- **Flag** — 标记对象可能有两种不同的行为方式
- **Strategy** — 不同的计算方式
- **State** — 单独抽象的状态对象
- **Component** — 整体与部分关系

Helper 案例

```
class Restaurant {  
    Courier courier;  
    public void takeOut() {  
        courier.drive();  
    }  
}
```

Flag 案例

- 现在要做这样一个事：
- `if(页数>1)` 则显示分页
- `else` 显示不分页
- 而分页(headview and footview)是动态加载进来的，且一个列表只能有一个 headview 和 footview。所以当我判断页数大于1时，headview and footview is added to listview。当我点击下一页时，它又会去加载headview和footview。我们要避免它重复的去加载。

- 我们可以设置一个标签去判断是否已经有headview 和 footview 了。
- 看代码：

```
private int mPage;  
private boolean flag = false;  
if (flag == false && mPage > 1) {  
    flag = true;  
    mListView.addHeaderView(mHeaderView);  
    mListView.addFootView(mFootView);  
}
```

Component 案例

```
class Segment {  
    Point a;  
    Point b;  
  
    public Segment(x1, y1, x2, y2) {  
        a = new Point(x1, y1);  
        b = new Point(x2, y2);  
    }  
}
```

总结

变量与数据类型

- └─ 为什么需要变量：状态 → 赋值 → 变量
- └─ 赋值的代价
 - | 引用透明性 · 环境模型 · 函数式 vs 命令式
- └─ 类型系统
 - | Strong/Weak × Static/Dynamic · Java类型层次
 - | 陷阱：溢出 · 浮点NaN · 类型提升
 - | 安全网：编译期/运行期/泛型
- └─ 变量属性
 - | 名字 · 地址(别名) · 生存期(栈/堆/GC) · 作用域 · 默认值
- └─ 变量角色
 - | 局部：收集器/计数/解释/复用/元素
 - | 属性：Helper/Flag/Strategy/State/Component

- 下一讲：控制流与方法——结构化编程的行为层面